



南開大學
Nankai University

南开大学

网络空间安全学院

《数据安全》课程作业

基于 zkSNARK 的零知识证明实验

姓名：_____

学号：_____

专业：_____

指导教师：_____

2026 年 4 月 21 日

目录

一、 实验要求	1
二、 实验仓库与目录结构	1
三、 实验原理	1
(一) 零知识证明的基本思想	1
(二) zkSNARK 与 libsnark	2
(三) 算术电路与 R1CS 约束	2
(四) 默认示例的见证计算	3
四、 实验过程	3
(一) 实验环境与工作目录设计	3
(二) conda 环境与 libsnark 环境初始化	4
(三) 证明代码实现	5
(四) 关键代码展示	5
(五) 编译流程	7
(六) 正确样例运行流程	8
(七) 错误样例运行流程	10
五、 实验结果与分析	11
(一) 结果文件	11
(二) 正确样例分析	12
(三) 负样例分析	12
六、 实验心得与体会	13

一、 实验要求

本次实验题目为“零知识证明实践”。需要参考教材第四章及实验 3.1 的思路，完成如下命题的证明生成与验证：

$$x^3 + x + 5 = Out.$$

其中， Out 为公开输入，默认取值为 35；证明者持有私密解 x ，在默认演示中使用 $x = 3$ 。实验目标是让验证者相信证明者知道一个满足约束的解，同时不直接暴露该解本身。

结合本次实现，实验边界进一步明确如下：

- 证明系统采用教材第四章介绍的 zkSNARK 路线，底层使用算术电路与 R1CS 描述命题；
- 证明与验证通过 `setup / prove / verify` 三阶段完成；
- Windows 侧仓库仅保留提交所需代码、脚本、结果与报告，WSL 中维护独立的构建工作区；
- 若在 WSL 中使用 Python，则统一通过 `conda` 环境管理，不直接依赖系统 Python 安装实验辅助依赖。

二、 实验仓库与目录结构

本次实验的代码、脚本、结果与报告统一组织在本地课程目录下，目录划分如下：

- 实验/lab2/zk_lab/：libsnark 证明代码；
- 实验/lab2/scripts/：PowerShell 入口脚本与 WSL shell 脚本；
- 实验/lab2/results/：证明密钥、验证密钥、证明文件与日志；
- 实验/lab2/report/：LaTeX 报告；
- 实验/lab2/src/：截图清单与占位说明。

如果后续需要提交到 GitHub，仅需将 实验/lab2/ 整体上传即可。本报告默认以本地路径为准进行撰写与说明。

三、 实验原理

（一） 零知识证明的基本思想

零知识证明（Zero-Knowledge Proof, ZKP）允许证明者在不泄露秘密本身的前提下，使验证者相信某个公开命题成立。教材中给出的定义可以概括为：证明者知道一个使得公开谓词 $C(x) = 1$ 的秘密输入 x ，并向验证者证明“自己确实知道它”，而不是把该秘密直接交给验证者。

零知识证明通常强调以下三个性质：

- **完备性**：若证明者确实知道合法见证，验证应当以高概率成功；
- **可靠性**：若证明者不知道见证，则伪造证明应当非常困难；
- **零知识性**：验证者只能获得“命题成立”这一事实，而不能得到关于见证本身的额外信息。

(二) zkSNARK 与 libsnark

zkSNARK 是一类典型的非交互式零知识证明技术，特点是证明短、验证快且适合公开验证。教材示例中使用 libsnark 框架来完成开发，该框架提供了：

- 约束系统定义；
- 证明密钥与验证密钥生成；
- 证明构造；
- 证明验证。

在工程上，本实验遵循如下流程：

1. 将待证明命题转成算术电路；
2. 将算术电路写成 R1CS 约束；
3. 在 `setup` 阶段生成 `pk.raw` 和 `vk.raw`；
4. 在 `prove` 阶段使用见证构造 `proof.raw`；
5. 在 `verify` 阶段使用公开输入和验证密钥检查证明是否成立。

(三) 算术电路与 R1CS 约束

本实验证明的命题是

$$x^3 + x + 5 = Out.$$

为了使用 zkSNARK，需要将其拆成仅包含加法和乘法的中间变量形式。设中间量如下：

$$x_square = x \cdot x,$$

$$x_cube = x_square \cdot x,$$

$$sum_with_x = x_cube + x,$$

$$expr_out = sum_with_x + 5.$$

最后要求：

$$expr_out = Out.$$

因此，本实验对应的 R1CS 约束固定为表 1 所示。

编号	约束
C1	$x \cdot x = x_square$
C2	$x_square \cdot x = x_cube$
C3	$(x_cube + x) \cdot 1 = sum_with_x$
C4	$(sum_with_x + 5) \cdot 1 = expr_out$
C5	$expr_out \cdot 1 = Out$

表 1: 实验命题对应的 R1CS 约束

在本实现中：

- 公有输入只有一个：`out`；
- 私有见证包括：`x`、`x_square`、`x_cube`、`sum_with_x`、`expr_out`。

（四）默认示例的见证计算

当默认示例取 $x = 3$ 时，可得到：

$$x_square = 3^2 = 9, \quad x_cube = 3^3 = 27,$$

$$sum_with_x = 27 + 3 = 30, \quad expr_out = 30 + 5 = 35.$$

因此在默认演示下，公开输入确实可以设为 $Out = 35$ ，验证者只看到公开输入和证明文件，不需要获得 $x = 3$ 本身。

四、实验过程

（一）实验环境与工作目录设计

本实验同时使用 Windows 仓库与 WSL 独立工作区。这样做的目的是让提交内容与外部依赖隔离，避免把庞大的 `libsark` 源码树直接放入课程仓库。

Windows 仓库目录为：

Windows 侧仓库结构

```
1 lab2/  
2 |- zk_lab/  
3 |- scripts/  
4 |- results/  
5 |- report/  
6 |- src/  
7 - tests/
```

WSL 侧固定工作目录为：

WSL 侧工作目录

```
1 /home/eric/workspace/data-security-lab2/  
2 |- env/  
3 |- libsark-src/  
4 |- build/  
5 |- artifacts/  
6 - logs/
```

```

eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ ls -la "$WORKSPACE_ROOT"
total 28
drwxr-xr-x 7 eric eric 4096 Apr 21 10:14 .
drwxr-xr-x 4 eric eric 4096 Apr 21 17:49 ..
drwxr-xr-x 2 eric eric 4096 Apr 21 13:04 artifacts
drwxr-xr-x 5 eric eric 4096 Apr 21 13:03 build
drwxr-xr-x 2 eric eric 4096 Apr 21 13:02 env
drwxr-xr-x 3 eric eric 4096 Apr 21 12:57 libsnaark-src
drwxr-xr-x 2 eric eric 4096 Apr 21 13:04 logs
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ find "$WORKSPACE_ROOT" -maxdepth 1 -type d | sort
/home/eric/workspace/data-security-lab2
/home/eric/workspace/data-security-lab2/artifacts
/home/eric/workspace/data-security-lab2/build
/home/eric/workspace/data-security-lab2/env
/home/eric/workspace/data-security-lab2/libsnaark-src
/home/eric/workspace/data-security-lab2/logs

```

图 1: WSL 独立工作目录创建成功

(二) conda 环境与 libsnaark 环境初始化

考虑到用户要求“WSL 中如果使用 Python，则建议用 conda 管理环境”，本实验的初始化脚本在 WSL 中固定创建环境 `datasec-lab2-zk`。即使主证明程序使用 C++/libsnaark，辅助检查、日志整理和后续可扩展脚本也统一在该环境内运行。

PowerShell 入口命令如下：

初始化 WSL 环境

```
1 .\scripts\setup_wsl_env.ps1
```

该脚本内部会完成以下工作：

1. 修正 WSL 中异常的 HOME 环境变量；
2. 安装 `build-essential`、`cmake`、`git` 等系统依赖；
3. 安装 Miniconda；
4. 创建 `datasec-lab2-zk` 环境；
5. 优先尝试准备 Windows 侧缓存；若因 Windows 路径限制无法完整签出，则自动回退到 WSL 中直接克隆 `libsnaark_abc` 及其子模块。

本次实际运行完成后，`env/environment.txt` 中记录的关键环境信息如下：

WSL 环境摘要

```

1 OS_PRETTY_NAME=Ubuntu 22.04.5 LTS
2 GENERATED_AT=2026-04-21 18:05:57 CST
3 HOME=/home/eric
4 WORKSPACE_ROOT=/home/eric/workspace/data-security-lab2
5 CONDA_ROOT=/home/eric/miniconda3
6 CONDA_ENV_NAME=datasec-lab2-zk
7 CONDA_PYTHON=/home/eric/miniconda3/envs/datasec-lab2-zk/bin/python
8 PYTHON_VERSION=Python 3.10.20
9 GIT=/usr/bin/git
10 CMAKE=/usr/bin/cmake

```

```

11 CXX=/usr/bin/c++
12 LIBSNARK_ROOT=/home/eric/workspace/data-security-lab2/libsnark-src/
    libsnark-abc-master

```

```

eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ source /home/eric/miniconda3/etc/profile.d/conda.sh
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ conda activate datasec-lab2-zk
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ which python
/home/eric/miniconda3/envs/datasec-lab2-zk/bin/python
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$

```

图 2: conda 环境创建与激活

```

eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ cat "$WORKSPACE_ROOT/env/environment.txt"
OS_PRETTY_NAME=Ubuntu 22.04.5 LTS
GENERATED_AT=2026-04-21 18:05:57 CST
HOME=/home/eric
WORKSPACE_ROOT=/home/eric/workspace/data-security-lab2
CONDA_ROOT=/home/eric/miniconda3
CONDA_ENV_NAME=datasec-lab2-zk
CONDA_PYTHON=/home/eric/miniconda3/envs/datasec-lab2-zk/bin/python
PYTHON_VERSION=Python 3.10.20
GIT=/usr/bin/git
CMAKE=/usr/bin/cmake
CXX=/usr/bin/c++
LIBSNARK_ROOT=/home/eric/workspace/data-security-lab2/libsnark-src/libsnark-abc-master

```

图 3: libsnark 环境准备成功

(三) 证明代码实现

证明代码放在 `zk_lab/` 目录，核心文件如下：

- `common.hpp`: 定义曲线类型、约束系统、见证结构、序列化辅助函数；
- `zk_setup.cpp`: 生成证明密钥与验证密钥；
- `zk_prove.cpp`: 根据给定见证生成证明；
- `zk_verify.cpp`: 读取验证密钥与证明文件，完成验证。

`common.hpp` 中的实现重点是：先在 `protoboard` 上分配公有输入和私有变量，再按表 1 依次加入约束。这样在 `setup` / `prove` / `verify` 三个阶段都可以复用同一套电路定义。

(四) 关键代码展示

结合往年实验报告的写法，本报告补充展示三段最能体现本实验核心逻辑的代码，分别对应“见证计算与约束构造”“证明生成”“证明验证”三个环节。

关键代码 1: 见证计算与 R1CS 约束构造

```

1 inline WitnessValues build_witness(const long long x)
2 {
3     WitnessValues witness{};
4     witness.x = x;
5     witness.x_square = x * x;

```

```

6     witness.x_cube = witness.x_square * x;
7     witness.sum_with_x = witness.x_cube + x;
8     witness.expr_out = witness.sum_with_x + 5;
9     return witness;
10 }
11
12 template<typename Field>
13 inline libsark::protoboard<Field> build_protoboard(
14     const long long public_out,
15     const WitnessValues *witness = nullptr)
16 {
17     libsark::protoboard<Field> pb;
18     ...
19     pb.set_input_sizes(1);
20     pb.val(out) = Field(public_out);
21
22     pb.add_r1cs_constraint(libsark::r1cs_constraint<Field>(x, x, x_square));
23     pb.add_r1cs_constraint(libsark::r1cs_constraint<Field>(x_square, x,
24         x_cube));
25     pb.add_r1cs_constraint(libsark::r1cs_constraint<Field>(x_cube + x, 1,
26         sum_with_x));
27     pb.add_r1cs_constraint(libsark::r1cs_constraint<Field>(sum_with_x + 5,
28         1, expr_out));
29     pb.add_r1cs_constraint(libsark::r1cs_constraint<Field>(expr_out, 1, out)
30         );
31     ...
32 }

```

上述代码首先根据秘密输入 x 展开中间见证值，然后在 `protoboard` 上依次加入 5 条 R1CS 约束。其中 `pb.set_input_sizes(1)` 明确规定只有 `out` 属于公有输入，其余变量均作为私有见证参与证明。

关键代码 2：证明阶段的 witness 检查与 proof 生成

```

1  const auto witness = lab2::build_witness(x_value);
2
3  if (!lab2::witness_matches_statement(witness, out_value)) {
4      std::cerr << "Witness does not satisfy the public statement." << std::
5          endl;
6      return EXIT_FAILURE;
7  }
8
9  const auto pb = lab2::build_protoboard<lab2::FieldT>(out_value, &witness);
10 if (!pb.is_satisfied()) {
11     std::cerr << "Constraint system is not satisfied." << std::endl;
12     return EXIT_FAILURE;
13 }
14
15 const auto proof = libsark::r1cs_gg_ppzksnark_prover<lab2::ppT>(

```



```
15     pk,  
16     pb.primary_input(),  
17     pb.auxiliary_input());
```

这段代码体现了证明阶段的两个关键检查：其一是 witness 与公开命题是否一致，其二是完整约束系统是否满足。只有两项检查都通过后，程序才会调用 prover 生成 proof.raw。

关键代码 3：验证阶段的核心调用

```
1  const auto pb = lab2::build_protoboard<lab2::FieldT>(out_value, nullptr);  
2  const bool verified = libsnark::r1cs_gg_ppzksnark_verifier_strong_IC<lab2::  
    ppT>(  
3      vk,  
4      pb.primary_input(),  
5      proof);  
6  
7  std::cout << "Verification result: " << (verified ? 1 : 0) << std::endl;
```

验证阶段不再接触私有 witness，而是只使用验证密钥、公开输入和证明文件完成检查。这也是本实验能够体现“公开验证而不泄露秘密”的关键所在。

（五） 编译流程

编译命令如下：

编译证明程序

```
1  .\scripts\build_lab2.ps1
```

该脚本会：

1. 将 zk_lab/ 中的源码同步到 WSL 工作区中的 libsnark_abc-master/src/;
2. 在外部 bundle 的 src/CMakeLists.txt 中追加 zk_setup、zk_prove、zk_verify 三个目标;
3. 调用 CMake 完成编译;
4. 将生成日志拷回 results/logs/build.log。

本次编译成功后，WSL 中生成的可执行文件为：

编译生成的二进制文件

```
1  /home/eric/workspace/data-security-lab2/build/src/zk_setup  
2  /home/eric/workspace/data-security-lab2/build/src/zk_prove  
3  /home/eric/workspace/data-security-lab2/build/src/zk_verify
```

```

eric@LAPTOP-BIQVW3B8:~/workspace/lab2$ sh "$REPO_ROOT/scripts/wsl_build_lab2.sh" "$REPO_ROOT" "$WORKSPACE_ROOT"
[lab2] Updating libsnark_abc submodules
Synchronizing submodule url for 'depends/libsnark'
Synchronizing submodule url for 'depends/libsnark/depends/ate-pairing'
Synchronizing submodule url for 'depends/libsnark/depends/gtest'
Synchronizing submodule url for 'depends/libsnark/depends/libff'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/ate-pairing'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/xyak'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/libff'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/ate-pairing'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/gtest'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/libff'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/ate-pairing'
Synchronizing submodule url for 'depends/libsnark/depends/libff/depends/xyak'
Synchronizing submodule url for 'depends/libsnark/depends/libsnark-supercop'
Synchronizing submodule url for 'depends/libsnark/depends/xyak'
[lab2] Syncing lab2 C++ sources into /home/eric/workspace/data-security-lab2/libsnark-src/libsnark_abc-master/src
[lab2] CMakeLists already contains lab2 target block
[lab2] Configuring and building libsnark bundle
[lab2] Build completed. Log saved to /home/eric/workspace/data-security-lab2/logs/build.log

```

(a) WSL 中执行编译脚本并输出完成提示

```

[lab2] Configuring and building libsnark bundle
[lab2] Build completed. Log saved to /home/eric/workspace/data-security-lab2/logs/build.log
eric@LAPTOP-BIQVW3B8:~/workspace/lab2$ ls -l "$WORKSPACE_ROOT/build/src"/zk.*
-rwxr-xr-x 1 eric eric 4285632 Apr 21 18:18 /home/eric/workspace/data-security-lab2/build/src/zk_prove
-rwxr-xr-x 1 eric eric 4483352 Apr 21 18:18 /home/eric/workspace/data-security-lab2/build/src/zk_setup
-rwxr-xr-x 1 eric eric 2848168 Apr 21 18:18 /home/eric/workspace/data-security-lab2/build/src/zk_verify
eric@LAPTOP-BIQVW3B8:~/workspace/lab2$

```

(b) 编译后生成的三个可执行文件

图 4: 编译阶段的终端输出与产物检查

(六) 正确样例运行流程

默认演示采用公开输入 $Out=35$ 和私密输入 $x=3$ 。PowerShell 侧运行命令为:

运行正确样例

```
1 .\scripts\run_demo.ps1 -X 3 -Out 35
```

内部执行顺序为:

1. `zk_setup --artifacts-dir ... --out 35`
2. `zk_prove --artifacts-dir ... --x 3 --out 35`
3. `zk_verify --artifacts-dir ... --out 35`

生成的主要文件包括:

- `results/artifacts/pk.raw`
- `results/artifacts/vk.raw`
- `results/artifacts/proof.raw`
- `results/logs/setup.log`
- `results/logs/prove_valid.log`
- `results/logs/verify_valid.log`

根据本次真实运行日志, 可以提取出以下关键结果:

setup/prove/verify 关键输出摘录

```

1 === Setup ===
2 Constraint count: 5
3 Public input size: 1
4 Saved proving key to: /home/eric/workspace/data-security-lab2/artifacts/pk.
   raw
5 Saved verification key to: /home/eric/workspace/data-security-lab2/artifacts/
   vk.raw
6
7 === Prove ===
8 Witness x = 3

```

```
9 x_square = 9
10 x_cube = 27
11 sum_with_x = 30
12 expr_out = 35
13 Primary input:
14 35
15 Auxiliary input:
16 3
17 9
18 27
19 30
20 35
21 Saved proof to: /home/eric/workspace/data-security-lab2/artifacts/proof.raw
22
23 === Verify ===
24 Verification result: 1
```

```
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ sh "$REPO_ROOT/scripts/wsl_run_demo.sh" "$REPO_ROOT"
"$WORKSPACE_ROOT" 3 35
[lab2] Valid demo finished
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$
```

图 5: 正确样例脚本执行完成

```
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$ grep -E 'Constraint count:|Public input size:|Saved proving key to:|Sa
ved verification key to:' "$REPO_ROOT/results/logs/setup.log"
Constraint count: 5
Public input size: 1
Saved proving key to: /home/eric/workspace/data-security-lab2/artifacts/pk.raw
Saved verification key to: /home/eric/workspace/data-security-lab2/artifacts/vk.raw
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$
```

图 6: zk_setup 生成密钥

```
Witness x = 3
x_square = 9
x_cube = 27
sum_with_x = 30
expr_out = 35
Primary input: 1
Auxiliary input: 5
Saved proof to: /home/eric/workspace/data-security-lab2/artifacts/proof.raw
eric@LAPTOP-BIQVRMJO:~/workspace/lab2$
```

图 7: zk_prove 成功生成证明

```
(leave) Call to r1cs_gg_ppzksnark_onli
start)
(leave) Call to r1cs_gg_ppzksnark_verifi
Verification result: 1
eric@LAPTOP-BIQVRMJ0:~/workspace/lab2$
```

图 8: zk_verify 验证结果为 1

(七) 错误样例运行流程

为了验证系统的可靠性，本实验还执行两个负样例：

1. 错误 witness: 令 $x=4$ 、 $out=35$ ，则约束不成立，证明阶段应失败；
2. 错误公开输入: 先用 $x=3$ 、 $out=35$ 生成合法证明，再用 $out=36$ 验证，结果应失败。

PowerShell 入口命令如下：

运行负样例

```
1 .\scripts\run_negative_cases.ps1
```

本次实际运行的负样例日志摘要如下：

负样例关键输出摘录

```
1 === Prove ===
2 Public statement:  $x^3 + x + 5 = Out$ 
3 Out = 35
4 Witness x = 4
5 x_square = 16
6 x_cube = 64
7 sum_with_x = 68
8 expr_out = 73
9 Witness does not satisfy the public statement.
10
11 === Verify ===
12 Public statement:  $x^3 + x + 5 = Out$ 
13 Out = 36
14 Verification result: 0
```

```
eric@LAPTOP-BIQVRMJ0:~/workspace/lab2$ sh "$REPO_ROOT/scripts/wsl_run_negative_cases.sh"
"$REPO_ROOT" "$WORKSPACE_ROOT"
[lab2] Negative cases finished
eric@LAPTOP-BIQVRMJ0:~/workspace/lab2$
```

图 9: 负样例脚本执行完成

```
=== Prove ===  
Public statement:  $x^3 + x + 5 = \text{Out}$   
Out = 35  
Witness x = 4  
x_square = 16  
x_cube = 64  
sum_with_x = 68  
expr_out = 73  
Witness does not satisfy the public statement.
```

图 10: 错误 witness 被拒绝

```
(leave) Online pairing computations  
(leave) Call to r1cs_gg_ppzksnark_online_veri  
start)  
(leave) Call to r1cs_gg_ppzksnark_online_veri  
start)  
(leave) Call to r1cs_gg_ppzksnark_verifier_stru  
Verification result: 0  
eric@LAPTOP-BIQVRMJ0:~/workspace/lab2$
```

图 11: 错误公开输入验证为 0

五、实验结果与分析

(一) 结果文件

从 WSL 工作区回收到的仓库的结果文件位于:

- results/artifacts/
- results/logs/

这些文件分别对应 setup、prove、verify 三个阶段及其负样例运行日志, 便于直接引用到报告或后续答辩展示中。

文件	大小（字节）	说明
pk.raw	1209	proving key
vk.raw	1148	verification key
proof.raw	137	zkSNARK proof
build.log	63668	编译日志
setup.log	6970	setup 阶段日志
prove_valid.log	5754	正确 witness 证明日志
verify_valid.log	4835	正确公开输入验证日志
prove_invalid.log	176	错误 witness 日志
verify_wrong_out.log	4876	错误公开输入验证日志

表 2: 本次实验回收的实际结果文件

```

eric@LAPTOP-BIQVRM30:~/workspace/lab2$ ls -l "$REPO_ROOT/results/artifacts"
total 12
-rw-r--r-- 1 eric eric 1209 Apr 21 18:14 pk.raw
-rw-r--r-- 1 eric eric 137 Apr 21 18:14 proof.raw
-rw-r--r-- 1 eric eric 1148 Apr 21 18:14 vk.raw
eric@LAPTOP-BIQVRM30:~/workspace/lab2$ ls -l "$REPO_ROOT/results/logs"
total 188
-rw-r--r-- 1 eric eric 63668 Apr 21 18:14 build.log
-rw-r--r-- 1 eric eric 176 Apr 21 18:14 prove_invalid.log
-rw-r--r-- 1 eric eric 5754 Apr 21 18:14 prove_valid.log
-rw-r--r-- 1 eric eric 6970 Apr 21 18:14 setup.log
-rw-r--r-- 1 eric eric 4835 Apr 21 18:14 verify_valid.log
-rw-r--r-- 1 eric eric 4876 Apr 21 18:14 verify_wrong_out.log
eric@LAPTOP-BIQVRM30:~/workspace/lab2$ python -m pytest -q "$REPO_ROOT/tests"
...
3 passed in 0.00s
eric@LAPTOP-BIQVRM30:~/workspace/lab2$

```

(a) 结果文件已从 WSL 工作区回收到仓库目录

(b) 在 conda 环境中执行 pytest, 测试全部通过

图 12: 实验结果文件检查与测试验证

(二) 正确样例分析

对于正确样例 $x=3$, $out=35$, 系统应呈现如下行为:

- setup 阶段生成 pk.raw 与 vk.raw;
- prove 阶段先检查 witness 与公开命题一致, 再构造证明 proof.raw;
- verify 阶段输出 Verification result: 1。

结合 setup.log、prove_valid.log 与 verify_valid.log 可知:

- 电路中的约束数量为 5, 与设计的 R1CS 约束完全一致;
- 公有输入规模为 1, 正好对应唯一公开量 Out;
- 正确 witness 展开后得到的辅助输入依次为 3, 9, 27, 30, 35;
- 最终验证输出为 1, 说明证明者确实可以在不公开 x 的情况下, 让验证者确认自己“知道一个使约束成立的见证”。

(三) 负样例分析

两个负样例分别说明:

- 若 witness 自身不满足约束, 则证明阶段就会被阻止, 不能继续生成合法证明;

- 即使已经存在一份合法证明，只要公开输入发生变化，验证仍会失败。

更具体地说：

- 错误 witness 的日志直接输出 `Witness does not satisfy the public statement.`;
- 错误公开输入的日志输出 `Verification result: 0`，并伴随 `QAP divisibility check failed.`。

因此，本实验同时展示了完备性与可靠性的直观效果。

场景	输入	预期结果
正确样例	$x = 3, Out = 35$	验证通过，输出 1
错误 witness	$x = 4, Out = 35$	证明阶段失败
错误公开输入	proof for (3, 35), verify with $Out = 36$	验证失败，输出 0

表 3: 实验样例与预期输出

六、实验心得与体会

通过本次实验，可以更加直观地认识到 zkSNARK 的工程实现并不是直接对原始数学公式做证明，而是需要先把命题拆解为算术电路，再进一步转化为标准化的 R1CS 约束系统。只有完成这种“问题表达方式”的转换，后续的 `setup` / `prove` / `verify` 三阶段流程才具有明确的输入、输出与验证语义。

与此同时，本实验也说明了实验环境规划对于课程作业的重要性。在 Windows 仓库、WSL 工作区、conda 环境和外部依赖源码并存的条件下，如果缺少统一的脚本入口与结果回收路径，日志、截图和中间产物很容易混乱。通过本次实现中对工作区、构建目录与结果目录的分离，可以显著提升实验复现性，也使最终报告整理更加规范。

总体而言，本实验将“零知识证明”从教材中的概念描述落实为一套可执行、可验证、可复现实验流程，对后续进一步学习范围证明、隐私计算证明以及通用电路证明具有较强的铺垫作用。